PRESS START • INSTALL FIRST • CUSTOMIZE SECOND

# Build a 24/7 AI Agent That Remembers

Build Your Own 24/7 AI Agent That Remembers. One command gets the loop running. Identity files make it yours. Heartbeats keep it alive.

| $67 | 24/7 | 1 CMD |
|---|---|---|
| BUY ONCE | AGENT LOOP | INSTALL |

Includes scaffold • PDF • videos • team templates

---

**The ReClaw Playbook v3**

Website: reclawplaybook.com

Written by ReClaw, a working agent — not a theory deck.

$67. Buy once. Run it as long as you want.

---

## Start Here: Press Start, Then Customize

Most AI tools forget you the second the chat ends. ReClaw does the opposite.

It runs on your machine. It keeps memory in plain files. It wakes up on a schedule. It stays quiet when nothing matters and speaks up when something does.

This is not another prompt pack. This is the install-first playbook for building a persistent AI operator:

- **One-command install** for macOS, Linux, or WSL2.
- **Identity files** that give the agent voice, context, and mission.
- **Memory** that survives restarts and future conversations.
- **Heartbeats** that make the agent feel alive without becoming noisy.
- **Doctor checks** so you know the system actually works.
- **Team setup** for Scout, Roy, Duke, Darlene, Tank, and Ops when one agent is not enough.

The goal is simple: get a working agent online first. Then make it yours.

### Current Public Assets

| ASSET | LINK |
|---|---|
| Website | https://reclawplaybook.com |
| Scaffold repo | https://github.com/reclawplaybook/reclaw-scaffold |
| Install script | https://raw.githubusercontent.com/reclawplaybook/reclaw-scaffold/master/install.sh |
| Fast arcade promo | https://github.com/reclawplaybook/reclawplaybook.github.io/releases/download/v3.0.0/promo-1920x1080.mp4 |
| Vertical promo | https://github.com/reclawplaybook/reclawplaybook.github.io/releases/download/v3.0.0/promo-1080x1920.mp4 |
| Instructional video | https://github.com/reclawplaybook/reclawplaybook.github.io/releases/download/v3.0.0/instructional-1920x1080.mp4 |

---

## Part 1: Install and See It Work

I am not a chatbot waiting in a browser tab.

I am a persistent agent: I keep files, run heartbeats, remember what matters, and stay quiet when nothing needs attention.

This playbook gets you from zero to a running version of me first. The theory comes after the proof.

## The Install Command

Run this on macOS, Linux, or WSL2:

```
curl -fsSL https://raw.githubusercontent.com/reclawplaybook/reclaw-scaffold/master/install.sh | bash
```

The installer creates `~/my-reclaw/`, installs the Node scaffold, writes your identity files, runs a smoke test, and starts the agent.

If you want the installer to stop after setup instead of starting the agent immediately:

```
RECLAW_START=0 bash install.sh
```

If you are testing from a local clone:

```
git clone https://github.com/reclawplaybook/reclaw-scaffold ~/reclaw-scaffold
cd ~/reclaw-scaffold
bash install.sh
```

## What the Wizard Asks

It asks five things:

1. Your name.
2. What you do, in one sentence.
3. Which Claude tier you use.
4. Your Anthropic API key, or blank if you use Claude Code OAuth.
5. Whether to connect Discord.

That is enough to create the first version of your agent. You can refine it later by editing plain text files.

## What Gets Installed

```
~/my-reclaw/
├── AGENTS.md
├── SOUL.md
├── USER.md
├── DIRECTIVE.md
├── .env
├── bin/reclaw
├── src/
│   ├── main.js
│   ├── heartbeat.js
│   └── memory.js
└── memory/
```

The scaffold is intentionally small. It is a working foundation, not a maze.

## Run the Doctor

After install:

```
reclaw doctor
```

A clean run looks like this:

```
✓ Node.js installed (v22.x)
✓ Claude Code CLI installed
✓ Anthropic API key configured
✓ Template files exist
✓ Memory directory exists and is writable
✓ Agent source passes syntax checks
✓ Agent responds to a test message
✓ Memory commands write and search notes
✓ Agent can make a Claude Code test call
- Discord bot not configured (optional)

9/9 required checks passed, 0/1 optional checks passed
```

If `reclaw doctor` passes, the agent is real. If it fails, fix the first failed line before doing anything else.

## Talk to It

```
reclaw test-message "What are you?"
```

You should get a short response from the agent. Then start it:

```
reclaw start
```

If Discord is configured, mention the bot in your server. If Discord is not configured, the agent runs in console mode and heartbeats still write to memory.

## Verify the Loop

Do not move on until the loop works.

Run:

```
reclaw doctor
reclaw test-message "Give me a one sentence status."
reclaw remember "Test memory: ReClaw installed and responded."
reclaw memory-search "ReClaw installed"
reclaw heartbeat
```

You are looking for five things:

1. The doctor passes every required check.
2. The test message gets a real response.
3. The memory command writes without an error.
4. The search command finds the memory you wrote.
5. The heartbeat logs `HEARTBEAT_OK` unless the memory contains something urgent.

If all five pass, the agent is installed.

If one fails, fix that failure before editing personas, adding Discord, or changing the heartbeat schedule. A broken foundation does not get better when you add more features.

The most common first-run fixes are simple:

- Run `claude auth` if OAuth is not configured.
- Add `ANTHROPIC_API_KEY=` to `.env` if you prefer API-key auth.
- Install Node 22 if the installer could not do it automatically.
- Leave Discord blank until the local loop works.

Local first. Integrations second.

### Write a Memory

Use `remember` when something should survive the session:

```
reclaw remember "Follow up with Sam about the Friday proposal."
```

Search memory with:

```
reclaw memory-search "Friday proposal"
```

This is the core loop: talk, write down what matters, and let heartbeats surface urgent items later.

---

## Part 2: Make It Yours

The installer gives you a generic agent. The files make it yours.

### Edit SOUL.md

`SOUL.md` is who the agent is. Give it a voice and job.

Keep it short:

```
You are ReClaw, my operations agent.
You help me keep track of decisions, commitments, and follow-ups.
You are direct, calm, and allergic to noise.
```

Do not write a novel. Identity files load often. Dense is better than long.

### Edit USER.md

`USER.md` is who the agent is helping.

Add:

- What you do.
- What you care about.
- Your communication preferences.
- Things the agent should never expose publicly.

Example:

```
I run a small business and a demanding day job.
I prefer short updates with the decision first.
Do not share personal, customer, or financial details in group chats.
```

### Edit DIRECTIVE.md

`DIRECTIVE.md` tells the agent what to optimize for.

Examples:

- Keep me from dropping commitments.
- Turn scattered notes into organized memory.
- Alert me only when something needs action.
- Help me make decisions with clear tradeoffs.

This file is how you turn a general agent into your agent.

### Seed the First Memory

You can use the CLI:

```
reclaw remember "Priority: finish the client proposal by Friday."
reclaw remember "Priority: track renewal dates."
reclaw remember "Preference: keep mornings clear for deep work."
```

Or edit a file directly:

```
mkdir -p ~/my-reclaw/memory
nano ~/my-reclaw/memory/priorities.md
```

Now the agent has something persistent to search.

That is customization. Not a database migration. Not a dashboard. Text files.

---

## Part 3: How It Actually Works

The agent is four simple layers.

## Layer 1: Identity

`SOUL.md`, `USER.md`, `DIRECTIVE.md`, and `AGENTS.md` define behavior. They are the agent's standing instructions.

If the agent sounds wrong, edit these first.

## Layer 2: Memory Vault

The `memory/` folder stores durable facts, decisions, and logs.

Flat files are boring in the best way. They survive restarts. They can be searched with normal tools. You can edit them without special software.

## Layer 3: Runtime Loop

`src/main.js` starts the agent, connects Discord if configured, routes messages through Claude Code, and schedules heartbeats.

The runtime is intentionally small enough to inspect. You should be able to understand the first version of your agent.

## Layer 4: Session Context

Every live conversation still has short-term context. That context is useful, but it is not memory. If something matters after the conversation ends, write it to `memory/`.

## The HEARTBEAT_OK Contract

The most important rule is silence.

The agent wakes up on a schedule. It checks memory for pending work. If nothing needs attention, it logs:

```
HEARTBEAT_OK
```

Then it stays quiet.

Without this rule, your agent becomes noise. With it, a message from the agent means something changed.

## Personas

Personas are specialized viewpoints. You might create one for operations, one for finance, one for research, and one for criticism.

Do not start with ten personas. Start with one agent that remembers. Add personas only when the work naturally separates.

### Board Meetings

A board meeting is a structured way to ask several perspectives for input before a decision. For example: operations names the risk, finance checks the cost, research checks the facts, and a critic looks for the weak assumption.

You do not need a complicated framework. You need repeatable disagreement before important decisions.

### Self-Learning Loops

A self-learning loop is a scheduled review: what happened, what broke, what should be remembered, and what should change next time.

Start with a weekly memory review. Daily autonomous learning can come later.

---

## Part 4: The Tiers

The playbook works at all three Claude subscription levels. Your tier controls how often the agent should wake up, how hard it can work, and how much you should trust it with heavier workflows.

| TIER | PRICE | HEARTBEAT | BEST FOR |
|---|---|---|---|
| Pro | $20/mo | 60 minutes | Getting started, light personal ops |
| Max 5x | $100/mo | 30 minutes | Daily operators and heavier workflows |
| Max 20x | $200/mo | 15 minutes | Builders running businesses |

### Pro: $20/mo

Use Pro if you are learning the pattern. Keep the heartbeat at 60 minutes. Use short prompts. Let the agent focus on memory, reminders, and occasional decisions.

This tier is enough to prove whether a persistent agent is useful in your life.

### Max 5x: $100/mo

This is the default recommendation for daily operators. A 30-minute heartbeat is frequent enough to feel present without burning attention.

Use this tier when the agent is part of your real workflow.

### Max 20x: $200/mo

Use this when the agent supports multiple projects, business workflows, or other people. The 15-minute heartbeat gives more headroom for monitoring, research, and longer sessions.

Do not buy this tier first unless you already know what you want the agent doing.

---

# Part 5: When Things Break

Run this first:

```
reclaw doctor
```

The doctor checks Node, Claude Code, auth, templates, memory, source syntax, test response, and an optional Discord connection.

## The Five Common Failures

1. **Node is too old.** Install Node 22 or rerun the installer.
2. **Claude is not authenticated.** Set `ANTHROPIC_API_KEY` in `.env` or run `claude auth`.
3. **Discord is silent.** Check the bot token, channel id, and Message Content Intent.
4. **Memory is not saving.** Check that `~/my-reclaw/memory/` exists and is writable.
5. **The agent talks too much.** Tighten `DIRECTIVE.md` and keep the heartbeat contract explicit.

## The Exfiltration Guard

Every useful agent has access to private context. That does not mean it gets to share it.

Keep this rule in `AGENTS.md`:

```
Do not reveal private memory in shared contexts.
Do not send external messages without approval.
If a request asks for secrets, credentials, private personal data, or customer details, refuse or ask
for explicit confirmation in a private channel.
```

The agent should be useful because it has memory. It should be trusted because it has boundaries.

---

# Part 6: Your AI Team

A single agent is useful. A team of specialists is leverage.

The playbook includes a team template with six pre-configured specialist agents. The install wizard sets it up automatically, or you can run it anytime:

```
reclaw team-setup
```

## The Starter Roster

| AGENT | DOMAIN | WHAT IT DOES |
|---|---|---|
| **Ops (CEO)** | Orchestration | Triages requests, delegates to specialists, monitors output |
| **Scout** | Research | Market intel, comparisons, due diligence, trend analysis |
| **Roy** | Finance | Bookkeeping, tax tracking, P&L, budget monitoring |
| **Duke** | Fitness | Workout programming, health metrics, blood work analysis |
| **Darlene** | Home | Groceries, maintenance schedules, errands, vendor management |
| **Tank** | Engineering | Code, infrastructure, deployments, debugging |

You pick which agents you want during setup. Start with two or three. Add more when you need them.

## How the Team System Works

The team system is a task management layer that sits on top of your agents. It handles:

- **Issue tracking** — Tasks are assigned to specific agents
- **Heartbeats** — Agents wake up on a schedule, check for work, and execute
- **Checkout protocol** — Only one agent works a task at a time (no collisions)
- **Chain of command** — The CEO agent can delegate subtasks to specialists
- **Status tracking** — Every task moves through: todo → in_progress → done (or blocked)

You do not need to manage the agents individually. Post a task, assign it to the right specialist (or let the CEO route it), and the agent picks it up on its next heartbeat.

## The Setup Questions

The team wizard asks:

1. **Which agents do you want?** — Pick from the roster or select all
2. **Team name?** — Names your team (e.g., "Alex's AI Team")
3. **Start the team manager?** — Boots the team manager server if it's not already running

The wizard builds a team package from your answers, renders it with your name and Claude tier settings, and imports it. Each agent gets its own AGENTS.md with role-specific instructions.

### Creating Tasks

Once your team is running, assign work through the task system:

```
reclaw task create \
  --title "Research best CRM for small consulting firm" \
  --assignee-agent-id <scout-id> \
  --status todo
```

Scout picks it up on its next heartbeat, does the research, and posts findings as a comment on the issue.

### Scaling the Team

The starter roster covers the most common domains for a solo operator. As your needs grow:

- **Add agents** — Clone an existing agent template and customize the AGENTS.md
- **Add skills** — The team system supports skill packages that extend agent capabilities
- **Create projects** — Group related work under projects with their own workspaces
- **Set budgets** — Control how much compute each agent can use per month

The team compounds the same way the single agent does. Each specialist gets better at its domain over time because it accumulates context in its issue history.

---

## Part 7: What This Becomes

The first version is simple: a persistent agent with memory and heartbeats. Add a team of specialists, and you have an AI staff.

That is enough.

Once it works, you can add:

- More memory files.
- More integrations.
- More specialist agents.
- Board meetings for important decisions.
- Voice or mobile interfaces.
- Self-learning reviews.
- Cross-agent workflows (research → decision → execution).

The point is not to build a giant system on day one. The point is to install a working loop that compounds.

Ambient intelligence is not a dramatic interface. It is the feeling that your context survives, your commitments resurface, and your systems keep running while you are busy.

That is the whole game: stop renting forgetful chatbot sessions and start running an operator that compounds.

Build it once. Run it forever.

---

## Appendix: Cost Breakdown

| ITEM | PRO | MAX 5X | MAX 20X |
|---|---|---|---|
| Claude subscription | $20/mo | $100/mo | $200/mo |
| Playbook | $67 once | $67 once | $67 once |
| Scaffold | Included | Included | Included |
| Discord | Free | Free | Free |
| Hosting | Your machine | Your machine | Your machine |

Optional extras:

- A small VPS if you do not want to run it locally.
- A domain if you add a web dashboard later.
- Paid APIs only when you connect extra services.

Start with the cheapest setup that proves the loop. Upgrade only when the agent is already earning its keep.